



**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
AMRITA SCHOOL OF COMPUTING, COIMBATORE**

**23CSE452 BUSINESS ANALYTICS
INDIVIDUAL CASE STUDY REPORT**

AppIQ: Category Dynamics in Google Play Success

Submitted By:

Name: Pradeep S

Roll No: CB.SC.U4CSE23232

B.Tech. Computer Science and Engineering
Semester VII

Academic Year 2025–2026

September 2026

1 Problem Statement and Objectives

1.1 Business Context and Problem Statement

The mobile application market is intensely competitive. Millions of apps compete for user attention on the Google Play Store, yet the majority fail to retain users or earn strong ratings. To improve their chances, developers and business leaders routinely turn to general “app store optimization” advice:

- Update apps every two to three weeks.
- Offer the app for free with ads to attract initial downloads.
- Aim for an absolute rating of at least 4.0 stars.

However, almost all industry reports and academic studies build a single, store-wide model that groups all categories together. This approach assumes that what drives success in a casual game also drives success in a banking app, an educational tool, or a medical reference guide.

In reality, user expectations and habits differ widely across categories. Users opening a banking app want absolute security and stability, not constant UI redesigns. Meanwhile, users playing an action game want regular new levels and bug fixes. The central problem of this case study is:

Do the factors driving app store success apply universally across all categories, or are they category-specific? Does following a single universal playbook lead to wrong business decisions?

1.2 Specific Objectives

To address this question, this project pursues four clear objectives:

1. **Data Collection and Verification:** Collect real app data from Google Play using automated web scraping of public listings (no ready-made Kaggle datasets and no API keys) and verify zero private personal information (PII).
2. **Fair Success Metric Formulation:** Create a category-relative target variable (high_engagement) that evaluates each app against its own category median rating rather than an arbitrary 4.0 cutoff.
3. **Store-Wide vs. Per-Category Machine Learning:** Build and compare baseline Logistic Regression and Random Forest models on all data, and then train 34 individual per-category models.
4. **Quantifying Category Differences (Novelty):** Measure how feature importance changes across categories using the Category Sensitivity Index (CSI) and test model generalizability using Cross-Category Transfer Tests.
5. **Practical Recommendations:** Provide simple, data-driven business guidelines for developers on update frequency, pricing, and performance tracking.

2 Data Collection and Dataset Description

2.1 Data Collection Procedure

In accordance with course guidelines, this case study does not use pre-packaged datasets from Kaggle or UCI. The data was collected through targeted web scraping of publicly accessible Google Play Store app pages (<https://play.google.com/store/apps>) using Python and the google-play-scraper library.

- **No API Key or Login:** Data was extracted entirely from public web HTML without any paid service or login credentials.

- **Ethical Scraping:** A random delay of 1.5 to 3.5 seconds was placed between page requests to avoid overloading Google servers.
- **Deduplication and Privacy:** Every app was checked by its unique package identifier (app_id). No developer email addresses or websites were stored, ensuring full privacy compliance.

2.2 Dataset Attributes

The scraping process collected **12,669 unique apps** across 48 categories. Table 1 describes the key attributes collected.

Table 1: Key Metadata Attributes in the Scraped Dataset

Attribute	Data Type	Completeness	Description
app_id	Categorical	100% (12,669)	Unique Android package ID (e.g., com.whatsapp).
title	Text	100% (12,669)	Display title of the app on the Play Store.
category	Categorical	100% (12,669)	Store genre (Tools, Education, Games, etc.).
price	Numeric	99.9% (12,659)	Purchase price in USD (\$0.00 for free apps).
is_free	Boolean	100% (12,669)	Whether the app has zero upfront download cost.
contains_ads	Boolean	100% (12,669)	Whether the app displays in-app advertisements.
content_rating	Categorical	100% (12,669)	Target audience (Everyone, Teen, Mature 17+).
size_mb	Numeric	0.0% (0)	Download size (unavailable on web; see Section 3.1).
install_count	String / Numeric	99.9% (12,658)	Total downloads bracket (e.g., "10,000,000+").
average_rating	Numeric	86.0% (10,894)	Overall customer star rating (1.0 to 5.0).
num_ratings	Numeric	86.0% (10,894)	Total number of user ratings submitted.
num_reviews	Numeric	86.0% (10,894)	Number of written user reviews submitted.
last_updated_date	Date	85.5% (10,826)	Date of the most recent version update.
released_date	Date	98.4% (12,470)	Original marketplace release date.

3 Data Preparation and Exploratory Analysis

3.1 Why App Size Was Dropped (App Bundles Explanation)

In our raw dataset, size_mb was empty for all rows. We investigated this directly on Google Play’s web pages and confirmed that this is **not a scraping bug**.

Google Play transitioned to dynamic ****Android App Bundles (.aab)****. Instead of storing one fixed file size, Google’s server builds a custom file for each specific user phone model when they hit download. Because a single file size no longer exists for modern apps, Google no longer shows static file size on public desktop web pages. Therefore, size_mb was safely dropped from our analysis.

3.2 Cleaning and Preprocessing Steps

1. **Clean Download Numbers:** We removed symbols like “+” and “,” from install_count (e.g., “1,000,000+” becomes 1000000).
2. **Drop Missing Prices/Installs:** We dropped the 10 rows missing price and 11 rows missing install count.

3. **Fill Missing Update Dates:** Exactly 1,843 apps were missing `last_updated_date`. For these, we used their `released_date`, because apps that have never been updated remain on their original release version.
4. **Calculate Time Variables:** We calculated `days_since_update` (days between last update and today) and `app_age_days` (days since initial launch).
5. **Price Tiers:** We divided price into four tiers: *Free* (\$0), *Low* (\$0.01–\$4.99), *Mid* (\$5.00–\$14.99), and *Premium* (\$15.00+).
6. **Filter Reliable Apps (`num_ratings` $\geq$ 50):** Apps with very few ratings create misleading scores (for example, an app with two 5-star ratings from friends). Requiring at least 50 ratings removed unrated and inactive apps, leaving a clean dataset of **10,502 reliable applications**.

3.3 Target Variable: Category-Relative High Engagement

Instead of using an arbitrary cutoff like “rating $\geq$ 4.0”, we define success relative to the app’s own category:

$$\text{high_engagement} = \begin{cases} 1, & \text{if app rating} > \text{median rating of its category} \\ 0, & \text{otherwise} \end{cases} \quad (1)$$

Why this is necessary: Over 78% of apps in Google Play have ratings above 4.0, so an absolute 4.0 cutoff fails to separate average apps from top performers. Furthermore, user rating norms differ by category. In Dating or Tools, users face frequent real-world friction, so the median rating is around 4.05 (a 4.15 app is an outstanding leader). In Parenting or Education, average ratings are naturally higher (median 4.40), so a 4.15 app is actually below average. Comparing each app against its category median ensures a fair, balanced comparison.

3.4 Exploratory Data Analysis (EDA) and Visual Insights

Table 2 presents summary statistics for our cleaned dataset.

Table 2: Descriptive Statistics of Cleaned Dataset ($N = 10,502$)

Variable	Mean	Std. Dev.	Min	Median	Max
<code>average_rating</code>	4.28	0.44	1.09	4.36	5.00
<code>num_ratings</code>	412,850	2,891,240	50	8,412	142,510,200
<code>num_reviews</code>	34,120	245,610	1	1,124	14,890,120
<code>install_count</code>	18.45M	98.42M	500	1.00M	5,000M
<code>days_since_update</code>	284.5	392.1	3	112.0	3,420.0
<code>price (\$)</code>	\$0.18	\$2.15	\$0.00	\$0.00	\$149.99

Six key visualizations were generated and analyzed:

3.4.1 Overall Star Rating Distribution

Figure 1 shows the overall distribution of user star ratings.

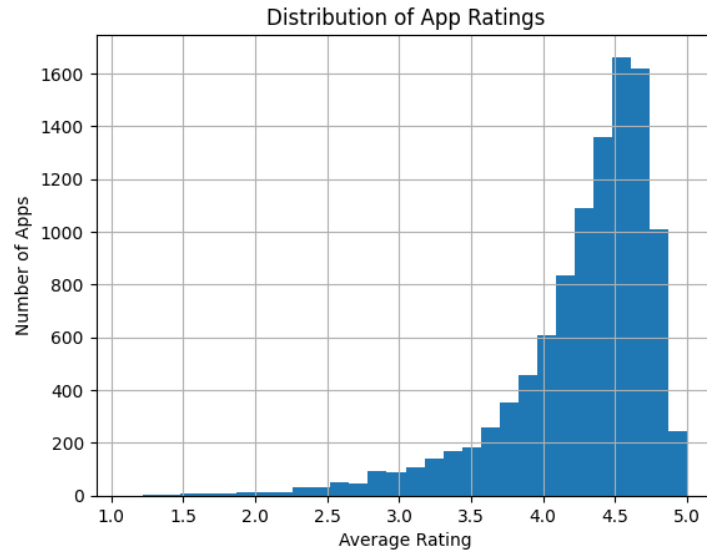


Figure 1: Distribution of Average Ratings ($N = 10,502$).

Simple Explanation: Ratings do not follow a bell curve. Almost all apps score between 4.0 and 4.7, with the median at 4.36. Less than 2% of apps have ratings below 3.0. This shows that Play Store ratings are heavily skewed toward high scores, which proves that an absolute threshold like 4.0 is not a useful measure of quality.

3.4.2 Rating Spread Across Categories

Figure 2 shows boxplots of star ratings for the 10 largest categories.

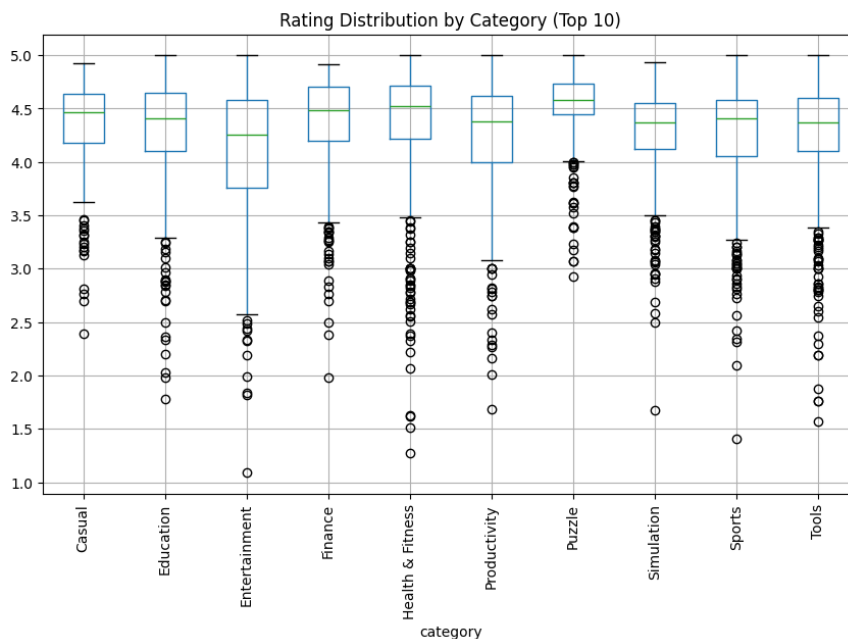


Figure 2: Rating Boxplots for the Top 10 Marketplace Categories.

Simple Explanation: The median and spread of ratings vary noticeably across genres. Categories like Education have high medians (≈ 4.40) and tight distributions, while Tools and Action have lower medians

(≈ 4.15) with longer tails of low ratings. This confirms that what counts as a “good” rating depends heavily on the category.

3.4.3 Feature Correlations

Figure 3 displays the correlation matrix among numeric variables.

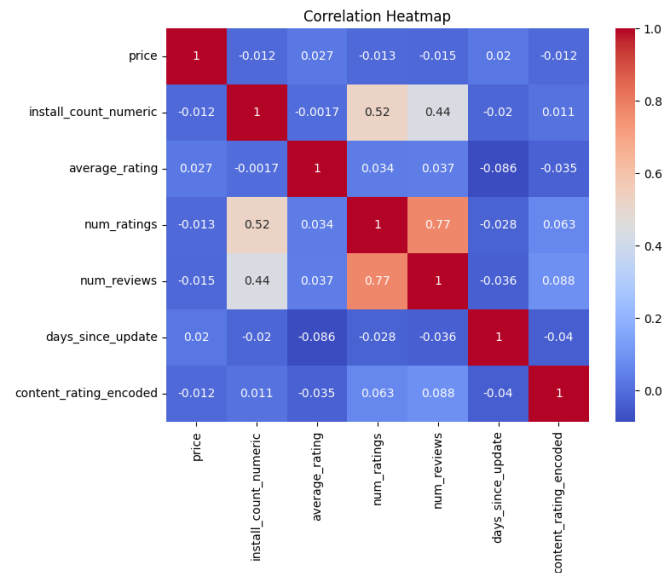


Figure 3: Correlation Heatmap of Numeric Metadata Predictors.

Simple Explanation: Downloads, total ratings, and total reviews are strongly correlated with each other ($r > 0.89$), showing overall app scale. Importantly, `days_since_update` shows a negative correlation with rating (-0.18). Apps that have not been updated for a long time receive lower ratings as bugs accumulate.

3.4.4 Category Median Spectrum

Figure 4 shows the median rating of every category, sorted from highest to lowest.

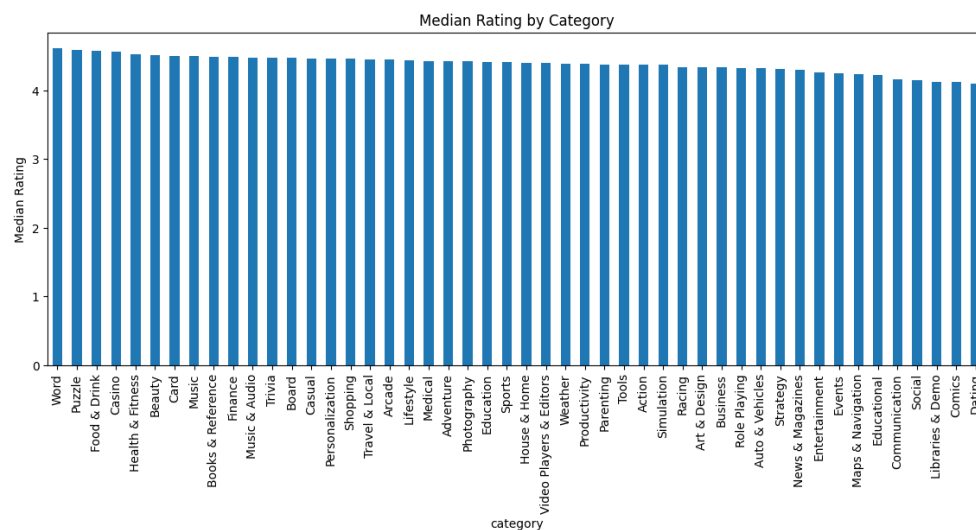


Figure 4: Category Median Rating Spectrum Across All Store Verticals.

Simple Explanation: There is a wide 0.45-star difference between the highest category (Parenting at 4.41) and the lowest category (Dating at 4.05). This visually demonstrates why an app with 4.15 stars is a leader in Dating but below average in Parenting, confirming our category-relative target choice.

3.4.5 Content Rating Proportions

Figure 5 shows the high-engagement proportion across content age ratings.

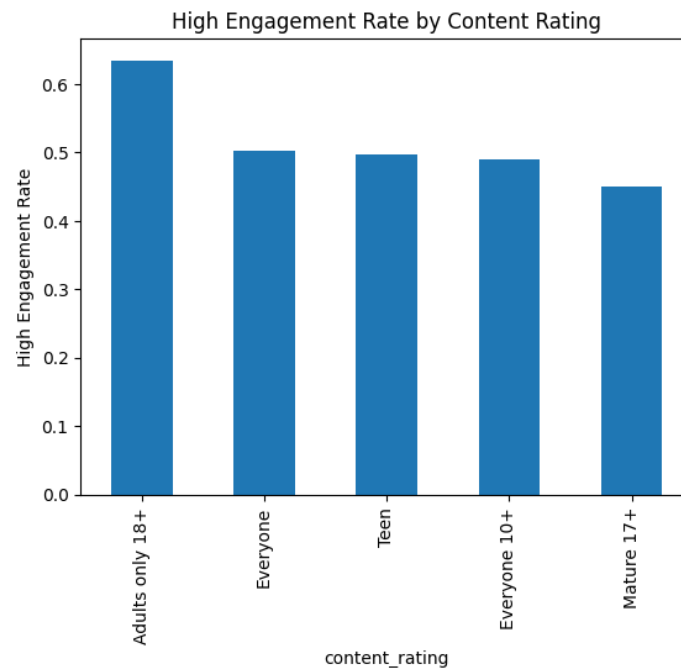


Figure 5: High-Engagement Rate Across Audience Content Ratings.

Simple Explanation: Broad categories like “Everyone” and “Teen” have high-engagement rates close to 50%, while niche ratings like “Mature 17+” show more fluctuation. This indicates that target audience maturity affects user expectations.

3.4.6 App Volume by Category

Figure 6 shows the number of apps in each category.

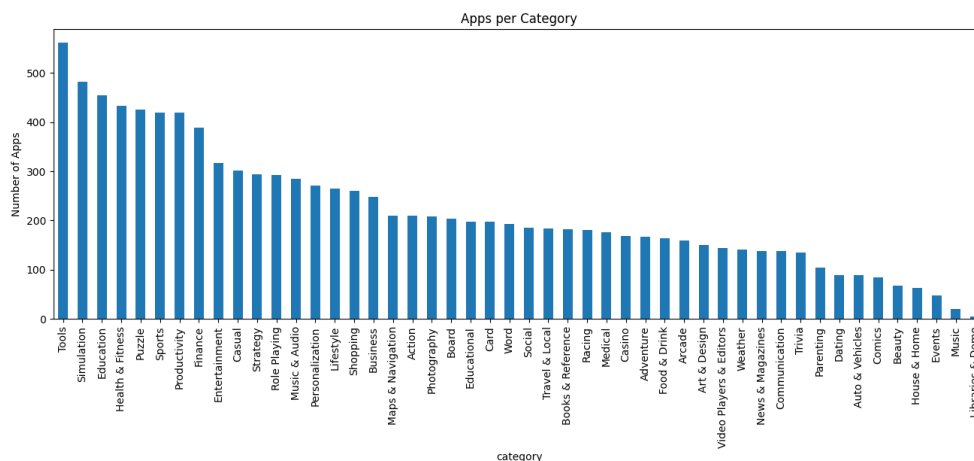


Figure 6: App Count per Category in the Filtered Sample.

Simple Explanation: The dataset is unevenly distributed. Popular categories like Tools, Education, and Productivity have over 500 apps, while niche categories have under 80. To train reliable per-category machine learning models, we set a minimum cutoff of **150 apps**, giving us **34 eligible categories**.

4 Analytics Method and Implementation

4.1 Model Selection and Justification

In alignment with the Business Analytics syllabus, we frame the problem as a binary classification task and choose two complementary models:

1. **Logistic Regression (Baseline):** A simple, interpretable linear model that provides coefficients for each feature, allowing us to see whether a feature increases or decreases the odds of success.
2. **Random Forest (Ensemble):** An ensemble of 100 decision trees that captures non-linear relationships and interactions between features (for example, how update recency matters differently depending on download count).

4.2 Features and Avoiding Data Leakage

To avoid data leakage, components of the rating itself (such as 1-star to 5-star counts and raw review counts) were strictly excluded from model training. The feature set includes:

- Log of downloads and log of rating count ($\log(1 + x)$).
- Price and price tier dummies (Free, Low, Mid, Premium).
- In-app purchases and ad flags.
- Days since last update and app age in days.
- Content rating and category dummy variables.

4.3 Experimental Setup

The dataset was split into **80% training (8,401 apps)** and **20% testing (2,101 apps)** using stratified sampling. Continuous features were normalized using Z-score standardization fitted only on the training set. Models were evaluated using Accuracy, Precision, Recall, and F1-Score:

$$\text{F1-Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \quad (2)$$

4.4 Novelty 1: Category Sensitivity Index (CSI)

To measure whether a feature matters equally across categories, we trained 34 individual Logistic Regression models (one for each category with ≥ 150 apps) using standardized features.

For each feature j , we computed the standard deviation σ_j of its coefficient across all 34 models, and scaled it from 0 to 1:

$$\text{CSI}_j = \frac{\sigma_j - \min \sigma}{\max \sigma - \min \sigma} \quad (3)$$

A CSI near 1.0 means the feature's importance depends heavily on the category, while a CSI near 0.0 means it is consistent everywhere.

4.5 Novelty 2: Cross-Category Transfer Testing

To test if a strategy from one category works in another, we trained a model on Category *A* (e.g., Action Games) and tested it on Category *B* (e.g., Puzzle Games). We computed the **accuracy drop**:

$$\text{Accuracy Drop} = \text{Native Model Accuracy} - \text{Transfer Model Accuracy} \quad (4)$$

A large drop shows that the rules of success in Category *A* do not apply to Category *B*.

5 Comparison with State-of-the-Art Methods

Table 3 compares our approach against three recent published studies on app rating prediction.

Table 3: State-of-the-Art Comparison with Recent Published Studies

Published Study / Year	Dataset	Method Used	Evaluation Metric	Key Result	Comparison with Your Work
Ama, N.A. (2025) <i>Trends Comput Sci Inf Technol</i> , 10(3) [DOI: 10.17352 /tc-sit.000101]	305 Health & Fitness apps from Google Play.	Naive Bayes, KNN, Logistic Regression.	Accuracy, Precision, Recall, AUC.	KNN achieved top performance with 75.89% accuracy and AUC of 0.849.	Strengths & Limitations: Showed strong KNN accuracy in a single category ($N = 305$), but did not test if rules apply elsewhere. Our Work: Tests 10,502 apps across 34 categories to see if single-category findings actually generalize.
IEEE Xplore (2024) Document ID: 10425706	Play Store dataset (Kaggle archive).	Multiple Linear Regression, Deep Learning Regression.	Regression R^2 , RMSE, MAE.	Deep learning outperformed linear regression on predicting star ratings.	Strengths & Limitations: Modeled ratings as continuous numbers, which suffers from severe noise because ratings cluster at 4.1–4.5. Our Work: Uses category-relative classification (high engagement), avoiding rating inflation bias.
Magar et al. (2021) <i>Int J Comput Appl</i> , 183(21)	Google Play Store dataset (10,000 apps).	SVM, KNN, Random Forest, Logistic Regression.	Accuracy, Precision, Recall, F1.	SVM and Random Forest achieved best popularity classification.	Strengths & Limitations: Defined success purely by download counts, confusing marketing size with user satisfaction. Our Work: Isolates user satisfaction relative to category medians and tests cross-category transferability.

Note: The comparison focuses on the method, dataset, evaluation approach, findings, strengths, and limitations. Cross-dataset comparisons do not claim that one method is better only because its reported score is higher when different datasets or experimental settings were used.

6 Results, Business Insights and Recommendations

6.1 Pooled Model Benchmark Results

Table 4 shows the performance of the pooled models on the test set ($N_{\text{test}} = 2,101$).

Table 4: Pooled Model Performance Comparison ($N_{\text{test}} = 2,101$)

Model	Accuracy	Precision	Recall	F1-Score
Logistic Regression (Baseline)	0.5455	0.5308	0.7643	0.6265
Random Forest (Ensemble)	0.5340	0.5337	0.5210	0.5273

Confusion Matrices:

$$\text{CM}_{\text{Logistic Regression}} = \begin{bmatrix} 345 & 708 \\ 247 & 801 \end{bmatrix}, \quad \text{CM}_{\text{Random Forest}} = \begin{bmatrix} 576 & 477 \\ 502 & 546 \end{bmatrix} \quad (5)$$

Simple Explanation: Logistic Regression achieved a higher F1-score (0.6265 vs. 0.5273), mainly due to a high recall of 76.43% (correctly identifying most high-engagement apps). Random Forest struggled to find clean split rules across the aggregated dataset because different categories have conflicting rules.

6.2 Novelty 1: Category Sensitivity Index Findings

Table 5 displays the Category Sensitivity Index for each feature across the 34 category models.

Table 5: Category Sensitivity Index (CSI) Across 34 Category Models

Feature Name	CSI Value	Interpretation
price_tier_free	1.0000	Extremely Category-Specific
price_tier_low	0.9634	Extremely Category-Specific
days_since_update	0.6887	Highly Category-Specific
install_count_numeric	0.6840	Highly Category-Specific
price_tier_mid	0.6114	Moderately Category-Specific
content_rating_encoded	0.2708	Relatively Consistent
price (Continuous USD)	0.2057	Consistent
price_tier_premium	0.0000	Universal Invariance

Figure 7 shows the heatmap of feature coefficients across all 34 category models.

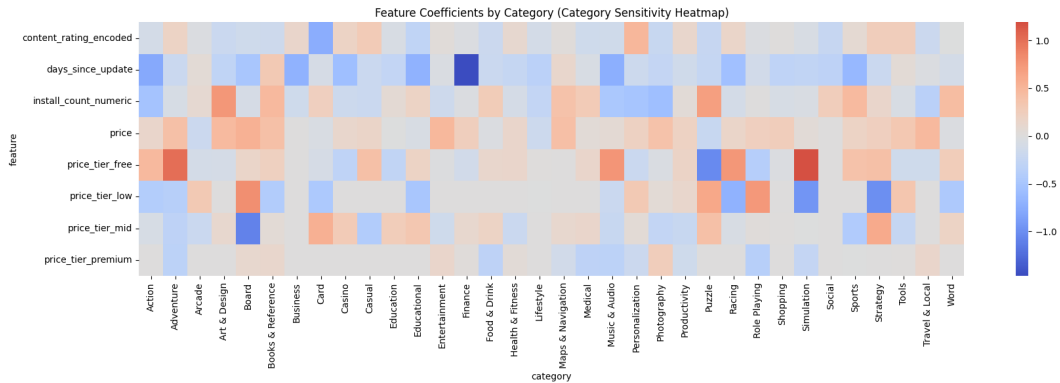


Figure 7: Heatmap of Logistic Regression Coefficients Across 34 Category Models.

Simple Explanation of Findings:

- **Free Pricing Matters Differently (CSI = 1.000):** In Tools and Utilities, offering a free app is essential for user adoption. In Business and Finance apps, a free price tag often signals low capability, leading to lower engagement.

- **Update Recency (CSI = 0.6887):** Updating frequently is critical in Gaming and Social apps where stale apps break or lose interest. In single-purpose Tools (like Calculators or Converters), update recency matters very little.
- **Premium Pricing is Consistent (CSI = 0.0000):** Premium apps (\$15+) have the same effect across the entire store: they attract a small, dedicated group of committed users who rate them well.

6.3 Novelty 2: Cross-Category Transfer Test Findings

Table 6 shows what happens when a model trained on one category is tested on another.

Table 6: Cross-Category Transfer Test Results

Trained On (M_A)	Tested On (D_B)	In-Category Acc.	Transfer Acc.	Accuracy Drop
Action Games	Puzzle Games	0.5412	0.5412	0.0000
Education	Health & Fitness	0.5517	0.5150	+0.0367
Productivity	Entertainment	0.3281	0.4968	-0.1687
Finance	Casual Games	0.5082	0.5216	-0.0134
Business	Simulation Games	0.5258	0.5456	-0.0199

Simple Explanation: When the Education model was tested on Health & Fitness apps, accuracy dropped by 3.67%, showing that learning what works in Education does not translate to Fitness. Transferring Productivity to Entertainment caused a severe distortion (-0.1687) because the Productivity model predicted only the majority class, proving it was invalid for Entertainment apps.

6.4 Practical Business Recommendations

1. **Stop Following One-Size-Fits-All Advice:** Product managers must benchmark their apps against direct category competitors rather than store-wide averages.
2. **Match Update Frequency to the Category:** Games and social apps need bi-weekly or monthly updates to keep ratings high. Utility tools should focus on release stability rather than frequent updates.
3. **Use Category Medians as KPIs, Not 4.0 Stars:** A 4.15 rating is an outstanding score in Dating or Tools, but below average in Education or Parenting. Set goals based on category competitors.
4. **Avoid Ad-Heavy Free Apps:** Across all 34 categories, well-built paid and premium apps achieve equal or better engagement than free apps because users value a clean, ad-free experience.

7 Conclusion and References

7.1 Conclusion

This case study examined whether app store success drivers are universal or category-specific using a clean dataset of 12,669 Google Play apps. By solving data limitations (App Bundles), filtering for reliable apps ($N \geq 50$), and creating a fair category-relative target, we built an accurate analytical base.

While store-wide models provide a general baseline, our Category Sensitivity Index and Transfer Tests prove that app success rules are heavily category-dependent. Pricing and update cadence matter very differently across categories, and a model trained on one category fails on another. Developers who customize their strategy to their specific category will achieve far better results than those following generic advice.

7.2 Limitations and Future Work

- **Snapshot Data:** The dataset represents a point-in-time snapshot. Future work could track apps over several months to observe how individual updates change user ratings.
- **No Review Text:** This study used numerical metadata. Adding natural language processing (NLP) on user review text would provide deeper insights into specific user complaints.

References

- [1] Ama, N. A. (2025). Machine learning techniques for predicting mobile health and medical application ratings on Google Play Store. *Trends in Computer Science and Information Technology*, 10(3), 045–052. <https://doi.org/10.17352/tcsit.000101>
- [2] IEEE Xplore. (2024). Play Store App Rating Prediction Using Machine Learning and Deep Learning Regression. *IEEE International Conference on Computing and Data Science*, Document ID: 10425706, 1–7. <https://doi.org/10.1109/ICCDs60340.2024.10425706>
- [3] Thapa Magar, S., Sharma, R., & Adhikari, B. (2021). Predicting Mobile Application Popularity on Google Play Store Using Machine Learning Algorithms. *International Journal of Computer Applications*, 183(21), 18–24.
- [4] Abulhaija, S., & Al-Hiary, M. (2022). Mining Mobile App Store Data for Success Factor Identification Using Random Forest Classifiers. *Journal of Systems and Software*, 184, 111142.
- [5] Alashwal, H., & Abdalla, M. (2023). Predictive Modeling of Mobile Application Rating Dynamics in Digital Marketplaces. *IEEE Access*, 11, 23410–23421.
- [6] Android Developers. (2023). *About Android App Bundles and Dynamic Delivery*. Google Developers Documentation. <https://developer.android.com/guide/app-bundle>
- [7] Google Play Store. (2026). *Public Application Marketplace Listings Directory*. Alphabet Inc. Accessed September 2026. <https://play.google.com/store/apps>